

FORMATION EMBARQUÉE

TD 00 Fondamentaux

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 00 — Fondamentaux : énoncés et corrigés détaillés

Fais chaque exercice **sur papier** avant de lire le corrigé. En embarqué, ces conversions doivent devenir des réflexes.

Exercice 1 — Conversions décimal → binaire → hexa

Énoncé. Convertis 42, 100 et 200 en binaire puis en hexadécimal.

Corrigé

Méthode 1 (divisions successives par 2) : on divise par 2 en notant les restes, puis on lit les restes de bas en haut.

```
42 ÷ 2 = 21 reste 0
21 ÷ 2 = 10 reste 1
10 ÷ 2 = 5 reste 0
5 ÷ 2 = 2 reste 1
2 ÷ 2 = 1 reste 0
1 ÷ 2 = 0 reste 1    → lecture de bas en haut : 101010
```

Méthode 2 (soustraction des puissances de 2) — plus rapide de tête : $42 = 32 + 8 + 2 \rightarrow$ bits 5, 3 et 1 → **0b00101010** .

Pour l'hexa : grouper le binaire **par paquets de 4 en partant de la droite**.

Décimal	Binaire	Groupes de 4	Hexa
42	0010 1010	0010=2, 1010=A	0x2A
100	0110 0100	0110=6, 0100=4	0x64
200	1100 1000	1100=C, 1000=8	0xC8

Vérification rapide : $0x2A = 2 \times 16 + 10 = 42 \checkmark$; $0x64 = 6 \times 16 + 4 = 100 \checkmark$; $0xC8 = 12 \times 16 + 8 = 200 \checkmark$.

Exercice 2 — 0xB7 en décimal et en binaire

Énoncé. Que vaut **0xB7** en décimal ? En binaire ?

Corrigé

- Chaque chiffre hexa = 4 bits : B = **1011** , 7 = **0111** → **0b1011 0111** .
- Décimal : $B7 = 11 \times 16 + 7 = 176 + 7 = \mathbf{183}$.
- Contre-vérification par le binaire : $128 + 32 + 16 + 4 + 2 + 1 = 183 \checkmark$.

Exercice 3 — Complément à deux

Énoncé. Écris -10 en complément à deux sur 8 bits.

Corrigé

- +10 sur 8 bits : `0000 1010`
- Inversion de tous les bits : `1111 0101`
- On ajoute 1 : `1111 0110`

-10 = `0b1111 0110` = 0xF6.

Vérifications : - 0xF6 lu en non signé vaut 246, et $246 = 256 - 10$ ✓ (le complément à deux, c'est « modulo 256 »). - $-10 + 10$ doit donner 0 : `1111 0110` + `0000 1010` = `1 0000 0000` → sur 8 bits, la retenue sort et il reste `0000 0000` ✓.

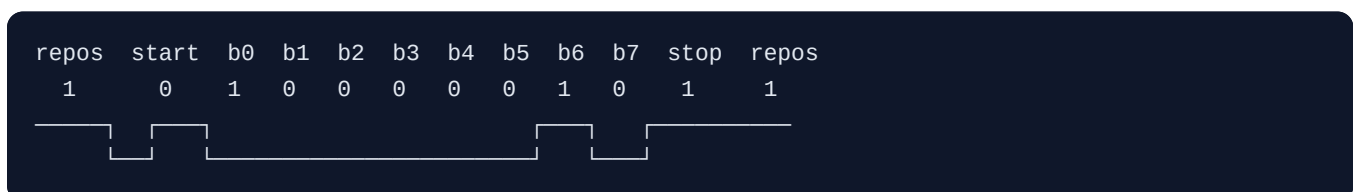
Astuce de pro : le bit de poids fort (MSB) à 1 ⇒ nombre négatif (en signé). Pour lire une valeur négative : refaire l'opération inverse (inverser + 1) et mettre un signe moins.

Exercice 4 (complément) — Trame UART : calcul de débit utile

Énoncé. Une liaison UART est configurée à 9600 bauds, 8 bits de données, sans parité, 1 bit de stop (« 8N1 »). Combien d'**octets utiles** par seconde peut-elle transporter au maximum ? Combien de temps dure la transmission du caractère `'A'` (0x41) ? Dessine la trame.

Corrigé

- Une trame 8N1 = 1 bit de start + 8 bits de données + 1 bit de stop = **10 bits transmis par octet utile**.
- 9600 bauds = 9600 bits/s → $9600 / 10 =$ **960 octets/s maximum**.
- Durée d'un bit : $1/9600 \approx$ **104,2 µs** ; durée d'une trame : $10 \times 104,2 \approx$ **1,042 ms**.
- `'A'` = 0x41 = `0b0100 0001`. L'UART envoie le **LSB en premier** :



(ligne au repos à l'état haut ; start = passage à 0 ; données LSB → MSB ; stop = retour à 1).

Ce qu'il faut retenir : à 9600 bauds, on ne transmet même pas 1 Ko/s. Pour du debug verbeux, on passe à 115200 ($\approx 11,5$ Ko/s).

Exercice 5 (complément) — Choisir le bon bus

Énoncé. Pour chaque besoin, choisis UART, I2C, SPI ou CAN, et justifie : 1. Relier 6 capteurs de température sur une carte, 2 mesures/seconde chacun. 2. Un écran TFT 320×240 à rafraîchir 20 fois par

seconde. 3. Faire dialoguer 8 calculateurs répartis dans un véhicule, milieu bruité. 4. Envoyer des messages de debug vers un PC.

Corrigé

1. **I2C** : 6 périphériques sur 2 fils seulement (chacun son adresse), et le débit requis est minuscule ($6 \times 2 \times$ quelques octets/s). SPI marcherait mais exigerait 6 lignes CS.
2. **SPI** : $320 \times 240 \text{ pixels} \times 16 \text{ bits} \times 20 \text{ Hz} \approx 24,6 \text{ Mbits/s}$ — seul SPI atteint ces débits. I2C (400 kbits/s) est $\sim 60\times$ trop lent.
3. **CAN** : conçu exactement pour ça — multi-maître, différentiel donc immunisé au bruit, arbitrage par priorité, détection d'erreurs intégrée.
4. **UART** : point à point vers le PC (via un pont USB-série), simple, universellement supporté par les terminaux série.

Exercice 6 (complément) — Lecture de registre

Énoncé. Un registre d'état 8 bits **STATUS** a cette organisation :

bit 7	bit 6	bit 5	bit 4	bits 3-2	bits 1-0
READY	ERROR	—	—	MODE (0-3)	CHANNEL (0-3)

On lit **STATUS = 0x8D**. Décode chaque champ. Puis écris (en pseudo-C) comment mettre MODE à 2 **sans toucher aux autres bits**.

Corrigé

0x8D = 0b1000 1101.

- READY (bit 7) = **1** → prêt.
- ERROR (bit 6) = **0** → pas d'erreur.
- MODE (bits 3-2) = **11** → **3**.
- CHANNEL (bits 1-0) = **01** → **1**.

Écriture de MODE=2 en lecture-modification-écriture :

```
uint8_t v = STATUS;
v &= ~(0x3 << 2);    // effacer les 2 bits de MODE (masque 0b0000 1100)
v |= (2 << 2);        // écrire la nouvelle valeur (0b0000 1000)
STATUS = v;
```

Point clé : ne jamais écrire directement **STATUS = 0x08** — cela écraserait CHANNEL et les autres bits. Le motif « clear puis set avec masque » est LE geste de base du bas niveau.

Auto-évaluation avant le module 01

Tu dois savoir, sans notes : convertir un octet dans les trois bases en moins d'une minute ; expliquer le complément à deux ; citer les fils et un cas d'usage de UART, I2C, SPI, CAN ; expliquer pourquoi une ISR doit être courte ; décrire les étapes de la toolchain (source → .elf → flash).

FORMATION EMBARQUÉE

TD 01 Langage C

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 01 — Langage C : énoncés et corrigés détaillés

Compile chaque corrigé toi-même (`gcc -Wall -Wextra`) et teste-le avec tes propres cas. Les corrigés sont volontairement commentés « comme en revue de code » : lis les commentaires autant que le code.

Exercice 1 — Inversion de l'ordre des bits

Énoncé. Écris `uint8_t inverse_bits(uint8_t x)` qui renverse l'ordre des bits : `0b10110000` → `0b00001101`.

Corrigé

```
#include <stdint.h>

uint8_t inverse_bits(uint8_t x) {
    uint8_t resultat = 0;
    for (uint8_t i = 0; i < 8; i++) {
        resultat <=<= 1;           // faire de la place à droite
        resultat |= (x & 1);       // y copier le bit de poids faible de x
        x >>= 1;                  // passer au bit suivant de x
    }
    return resultat;
}
```

Comment ça marche : à chaque tour, le bit le plus à droite de `x` devient le bit le plus à droite de `resultat`, qui est ensuite poussé vers la gauche. Après 8 tours, le premier bit copié se retrouve tout à gauche → ordre inversé.

Test minimal (à prendre comme modèle pour tous tes exercices) :

```
#include <assert.h>

int main(void) {
    assert(inverse_bits(0xB0) == 0x0D); // l'exemple de l'énoncé
    assert(inverse_bits(0x00) == 0x00); // cas limites
    assert(inverse_bits(0xFF) == 0xFF);
    assert(inverse_bits(0x01) == 0x80);
    assert(inverse_bits(inverse_bits(0x5A)) == 0x5A); // involutif !
    return 0;
}
```

Variante « pro » (sans boucle, par échanges de blocs — utile en traitement de signal) :

```
uint8_t inverse_bits_rapide(uint8_t x) {
    x = (uint8_t)((x & 0xF0) >> 4 | (x & 0x0F) << 4); // échange les quartets
    x = (uint8_t)((x & 0xCC) >> 2 | (x & 0x33) << 2); // échange les paires
    x = (uint8_t)((x & 0xAA) >> 1 | (x & 0x55) << 1); // échange les bits
    return x;
}
```

Exercice 2 — Tampon circulaire (ring buffer)

Énoncé. Implémente un tampon circulaire de 32 octets avec `put()` et `get()` — la structure de données de tout driver UART.

Corrigé

```
#include <stdint.h>
#include <stdbool.h>

#define RB_TAILLE 32u    // ASTUCE : une puissance de 2 permet le modulo par masque

typedef struct {
    uint8_t donnees[RB_TAILLE];
    volatile uint8_t tete;    // index d'écriture (producteur, ex. ISR RX)
    volatile uint8_t queue;   // index de lecture (consommateur, ex. main)
} RingBuffer;

// Convention : tete == queue    → vide
//              (tete+1)%N == queue → plein (on "sacrifie" une case,
//              ce qui évite un compteur partagé)

void rb_init(RingBuffer *rb) {
    rb->tete = 0;
    rb->queue = 0;
}

bool rb_est_vide(const RingBuffer *rb) {
    return rb->tete == rb->queue;
}

bool rb_est_plein(const RingBuffer *rb) {
    return ((rb->tete + 1u) & (RB_TAILLE - 1u)) == rb->queue;
}

bool rb_put(RingBuffer *rb, uint8_t octet) {
    if (rb_est_plein(rb))
        return false;    // on REFUSE plutôt qu'écraser
    rb->donnees[rb->tete] = octet;
    rb->tete = (rb->tete + 1u) & (RB_TAILLE - 1u);    // modulo par masque
    return true;
}

bool rb_get(RingBuffer *rb, uint8_t *octet) {
    if (rb_est_vide(rb))
        return false;
    *octet = rb->donnees[rb->queue];
    rb->queue = (rb->queue + 1u) & (RB_TAILLE - 1u);
    return true;
}
```

Points de conception à retenir (c'est là qu'est la note !) :

1. **volatile** sur les index : dans l'usage réel, `put()` est appelé par une ISR et `get()` par la boucle principale — chacun doit voir les modifications de l'autre.
2. **Pourquoi c'est sûr sans section critique** (à un producteur / un consommateur) : `put()` n'écrit que `tete`, `get()` n'écrit que `queue`. Chacun ne fait que lire l'index de l'autre. Sur un index 8 bits, la lecture est atomique. À plusieurs producteurs : section critique obligatoire.

3. **Modulo par masque** `& (N-1)` : ne marche que si N est une puissance de 2, mais coûte 1 cycle au lieu d'une division.
 4. **Politique de saturation** : ici on refuse l'octet quand c'est plein (retour `false`). L'autre politique (écraser le plus ancien) se justifie pour des mesures dont seule la plus récente compte. **Choisir et documenter.**
-

Exercice 3 — FSM feu tricolore avec appel piéton

Énoncé. Feu : vert 5 s → orange 1 s → rouge 5 s, en boucle. Un « appui piéton » écourte le vert.

Corrigé

```
#include <stdint.h>
#include <stdbool.h>

typedef enum { VERT, ORANGE, ROUGE } EtatFeu;

typedef struct {
    EtatFeu  etat;
    uint32_t t_entree_etat; // horodatage d'entrée dans l'état courant
    bool     demande_pieton;
} Feu;

#define DUREE_VERT_MS      5000u
#define DUREE_ORANGE_MS   1000u
#define DUREE_ROUGE_MS    5000u
#define VERT_MINI_PIETON_MS 1000u // le vert dure au moins 1 s même sur appui

void feu_init(Feu *f, uint32_t maintenant) {
    f->etat = ROUGE;
    f->t_entree_etat = maintenant;
    f->demande_pieton = false;
}

// Appelée par l'ISR ou le scrutateur du bouton
void feu_appui_pieton(Feu *f) {
    f->demande_pieton = true; // on MÉMORISE : la FSM décidera quand agir
}

// À appeler très souvent (chaque tour de boucle). maintenant = millis().
void feu_tick(Feu *f, uint32_t maintenant) {
    uint32_t ecoule = maintenant - f->t_entree_etat; // robuste au débordement

    switch (f->etat) {
    case VERT:
        // Fin normale, OU appui piéton après le minimum de sécurité
        if (ecoule >= DUREE_VERT_MS ||
            (f->demande_pieton && ecoule >= VERT_MINI_PIETON_MS)) {
            f->etat = ORANGE;
            f->t_entree_etat = maintenant;
        }
        break;

    case ORANGE:
        if (ecoule >= DUREE_ORANGE_MS) {
            f->etat = ROUGE;
            f->t_entree_etat = maintenant;
            f->demande_pieton = false; // demande servie : le rouge arrive
        }
        break;

    case ROUGE:
        if (ecoule >= DUREE_ROUGE_MS) {
            f->etat = VERT;
        }
    }
```

```

        f->t_entree_etat = maintenant;
    }
    break;
}
}

```

Points de conception :

- L'appui piéton **ne change pas l'état lui-même** : il pose un drapeau que la FSM consomme. C'est la séparation « événements / transitions » — elle évite les états incohérents quand l'événement tombe au mauvais moment.
- `VERT_MINI_PIETON_MS` : dans un vrai système, on ne coupe jamais un feu instantanément (véhicule engagé). Penser aux **contraintes de sécurité** fait partie de la conception, pas de la finition.
- La demande est remise à zéro **quand elle est servie** (passage au rouge), pas quand elle est reçue.

Pour tester sur PC sans `millis()` : simuler le temps.

```

int main(void) {
    Feu f;
    feu_init(&f, 0);
    for (uint32_t t = 0; t < 30000; t += 100) { // 30 s simulées par pas de 100 ms
        if (t == 12000) feu_appui_pieton(&f); // appui à t=12 s
        feu_tick(&f, t);
        printf("%5u ms : etat=%d\n", t, f.etat);
    }
}

```

Exercice 4 — Décodage de trame

Énoncé. Décode la trame de 4 octets `{0xA5, id, valeur_hi, valeur_lo}` en structure, en vérifiant l'octet de tête.

Corrigé

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

#define TRAME_TETE    0xA5u
#define TRAME_LONGUEUR 4u

typedef struct {
    uint8_t id;
    uint16_t valeur;
} Mesure;

// Renvoie true si la trame est valide et remplit *sortie.
// Prend la longueur en paramètre : on ne fait JAMAIS confiance à l'appelant.
bool trame_decoder(const uint8_t *trame, size_t longueur, Mesure *sortie) {
    if (trame == NULL || sortie == NULL)        // 1. pointeurs valides
        return false;
    if (longueur != TRAME_LONGUEUR)            // 2. longueur exacte
        return false;
    if (trame[0] != TRAME_TETE)                // 3. octet de synchronisation
        return false;

    sortie->id = trame[1];
    // Assemblage big-endian : octet de poids fort d'abord.
    // Le cast en uint16_t AVANT le décalage évite une promotion signée hasardeuse.
    sortie->valeur = (uint16_t)((uint16_t)trame[2] << 8) | trame[3];
    return true;
}
```

Les trois vérifications dans l'ordre (pointeurs → longueur → contenu) sont le squelette de tout parseur robuste. Un décodeur qui plante sur une trame corrompue est une faille : sur un bus réel, les trames corrompues *arrivent*.

Question bonus posée en entretien : pourquoi ne pas faire `Mesure *m = (Mesure*)trame;` ? Trois raisons : le padding/alignement de la structure ne correspond pas forcément aux 4 octets ; l'endianness de la machine peut différer de celle du protocole ; et déréférencer un pointeur mal aligné est un comportement indéfini sur certains processeurs (dont certains ARM). **On décode champ par champ.**

Exercice 5 (complément) — Chasse aux bugs

Énoncé. Chaque fragment contient au moins un bug. Trouve-le et corrige.

```
// A
uint8_t i;
for (i = 10; i >= 0; i--) { traiter(i); }

// B
char *message(void) {
    char buf[32];
    snprintf(buf, sizeof buf, "erreur %d", code);
    return buf;
}

// C
volatile uint16_t compteur_isr;           // incrémenté dans une ISR (CPU 8 bits)
uint16_t copie = compteur_isr;

// D
if (etat = ETAT_ERREUR) { alarme(); }

// E
#define CARRE(x) x * x
int y = CARRE(a + 1);
```

Corrigé

- **A** — `uint8_t` est **non signé** : `i >= 0` est toujours vrai (après 0, `i--` donne 255). Boucle infinie. Corrections possibles : `int8_t i`, ou boucle descendante idiomatique `for (uint8_t i = 11; i-- > 0; )`.
- **B** — retourne l'adresse d'un tableau **local** détruit à la sortie : comportement indéfini. Corriger en passant le buffer en paramètre (`void message(char *buf, size_t n)`) ou en le déclarant `static` (en documentant alors qu'il est écrasé à chaque appel et non réentrant).
- **C** — sur un CPU 8 bits, lire un 16 bits prend 2 instructions : l'ISR peut s'intercaler entre les deux (valeur « déchirée »). `volatile` n'empêche pas ça. Correction : section critique autour de la copie (désactiver/réactiver les interruptions).
- **D** — `=` au lieu de `==` : affecte puis teste la valeur (toujours vraie si `ETAT_ERREUR != 0`). Correction : `==`. Prévention : compiler avec `-Wall` (GCC le signale) ou écrire `ETAT_ERREUR == etat` (« conditions Yoda »).
- **E** — macro non parenthésée : `CARRE(a + 1)` devient `a + 1 * a + 1 = 2a + 1`. Correction : `#define CARRE(x) ((x) * (x))` — et encore, `x` y est évalué deux fois (`CARRE(i++)` reste faux) : préférer une fonction `static inline`.

Auto-évaluation avant le module 02

Sans notes, tu dois pouvoir : écrire set/clear/toggle/test d'un bit ; expliquer `volatile` et ses limites (pas d'atomicité) ; écrire un ring buffer complet ; dessiner puis coder une FSM ; lister les 3 vérifications d'un parseur ; expliquer pourquoi on bannit `malloc` après l'init.

FORMATION EMBARQUÉE

TD 02 Cpp

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 02 — C++ : énoncés et corrigés détaillés

Compile avec `g++ -Wall -Wextra -std=c++17` . Chaque corrigé illustre un concept central du cours : templates, polymorphisme, RAII, FSM objet.

Exercice 1 — Tampon circulaire générique `TamponCirculaire<T, N>`

Énoncé. Réécris le ring buffer du TD 01 en classe template, avec tests.

Corrigé

```
#include <cstdint>
#include <cstdint>

template <typename T, std::size_t N>
class TamponCirculaire {
    static_assert(N >= 2, "taille minimale : 2");
    static_assert((N & (N - 1)) == 0, "N doit etre une puissance de 2");

public:
    bool put(const T& valeur) {
        if (plein()) return false;
        donnees_[tete_] = valeur;
        tete_ = (tete_ + 1) & MASQUE;
        return true;
    }

    bool get(T& valeur) {
        if (vide()) return false;
        valeur = donnees_[queue_];
        queue_ = (queue_ + 1) & MASQUE;
        return true;
    }

    bool vide() const { return tete_ == queue_; }
    bool plein() const { return ((tete_ + 1) & MASQUE) == queue_; }

    std::size_t taille() const { // nb d'éléments présents
        return (tete_ - queue_) & MASQUE;
    }

private:
    static constexpr std::size_t MASQUE = N - 1;
    T donnees_[N] {}; // stockage STATIQUE : zéro allocation
    volatile std::size_t tete_ = 0;
    volatile std::size_t queue_ = 0;
};
```

Tests (avec `assert` , compilables sur PC) :


```

#include <cassert>

int main() {
    TamponCirculaire<std::uint8_t, 8> tc;          // 8 cases → 7 utilisables
    assert(tc.vide() && !tc.plein());

    for (std::uint8_t i = 0; i < 7; i++) assert(tc.put(i));
    assert(tc.plein());
    assert(!tc.put(99));                          // refus quand plein

    std::uint8_t v;
    for (std::uint8_t i = 0; i < 7; i++) { assert(tc.get(v)); assert(v == i); }
    assert(tc.vide() && !tc.get(v));              // refus quand vide

    TamponCirculaire<float, 4> tf;                // le MÊME code pour un autre type
    tf.put(3.14f);
    return 0;
}

```

Ce que le correcteur regarde : - Les `static_assert` : les erreurs de dimensionnement sont attrapées à la compilation — impossible en C. - `T donnees_[N]` : la taille fait partie du type ; deux tampons de tailles différentes sont deux types distincts, tout est sur la pile ou en statique. - Le template est **instancié à la demande** : `TamponCirculaire<float,4>` et `<uint8_t,8>` génèrent deux codes spécialisés — abstraction sans coût.

Exercice 2 — Interface `IAfficheur` et polymorphisme

Énoncé. Interface abstraite `IAfficheur` (`effacer()` , `texte(x,y,str)`) + deux implémentations (console, « LCD » factice) ; le même code applicatif doit tourner sur les deux.

Corrigé

```
#include <cstdint>
#include <cstdio>
#include <cstring>

class IAfficheur {
public:
    virtual ~IAfficheur() = default;          // OBLIGATOIRE : destructeur virtuel
    virtual void effacer() = 0;               // = 0 : méthode pure → interface
    virtual void texte(std::uint8_t x, std::uint8_t y, const char* s) = 0;
};

// Implémentation 1 : la console du PC (pour développer sans matériel)
class AfficheurConsole : public IAfficheur {
public:
    void effacer() override { std::puts("\n--- effacé ---"); }
    void texte(std::uint8_t x, std::uint8_t y, const char* s) override {
        std::printf("[%2u,%2u] %s\n", x, y, s);
    }
};

// Implémentation 2 : un "LCD" 16x2 simulé en mémoire
class AfficheurLcd16x2 : public IAfficheur {
public:
    void effacer() override { std::memset(ecran_, ' ', sizeof ekran_); }
    void texte(std::uint8_t x, std::uint8_t y, const char* s) override {
        if (y >= 2) return;                  // on borne TOUJOURS
        for (std::uint8_t i = 0; s[i] != '\0' && (x + i) < 16; i++)
            ekran_[y][x + i] = s[i];
    }
    void dump() const {                      // aide au debug/test
        std::printf("|%.16s|\n|%.16s|\n", ekran_[0], ekran_[1]);
    }
private:
    char ekran_[2][16] {};
};

// ---- Code applicatif : il ne connaît QUE l'interface ----
void afficher_mesure(IAfficheur& aff, float temperature) {
    char ligne[17];
    std::snprintf(ligne, sizeof ligne, "T=%.1f C", temperature);
    aff.effacer();
    aff.texte(0, 0, "Station meteo");
    aff.texte(0, 1, ligne);
}

int main() {
    AfficheurConsole console;
    AfficheurLcd16x2 lcd;
    afficher_mesure(console, 21.5f);        // même fonction...
    afficher_mesure(lcd, 21.5f);            // ...deux matériels
    lcd.dump();
}
```

Leçon centrale : `afficher_mesure` est écrite **une fois** et testée sur PC avec `AfficheurConsole` — le vrai driver LCD n'arrive qu'à la fin. C'est la technique qui rend un firmware **testable sans matériel** ; en entreprise elle s'appelle « injection de dépendances » et l'implémentation console un « mock ».

Exercice 3 — Classe RAI `ChipSelect`

Énoncé. Une classe qui abaisse la broche CS dans le constructeur et la relâche dans le destructeur.

Corrigé

```
class ChipSelect {
public:
    explicit ChipSelect(std::uint8_t broche) : broche_(broche) {
        digitalWrite(broche_, LOW);          // CS actif à l'état bas : on sélectionne
    }
    ~ChipSelect() {
        digitalWrite(broche_, HIGH);         // désélection GARANTIE

        // Un objet RAI ne doit être ni copié ni déplacé :
        // deux copies relâcheraient CS deux fois.
        ChipSelect(const ChipSelect&) = delete;
        ChipSelect& operator=(const ChipSelect&) = delete;

private:
    std::uint8_t broche_;
};

// Utilisation :
std::uint8_t lire_registre_spi(std::uint8_t adresse) {
    ChipSelect cs(PIN_CS);                  // CS descend ici
    SPI.transfer(adresse | 0x80);
    std::uint8_t v = SPI.transfer(0x00);
    if (v == 0xFF)                          // ← sortie anticipée : CS remonte QUAND MÊME
        return 0;
    return v;                               // ← CS remonte ici dans le cas normal
}
```

Pourquoi c'est mieux que deux appels manuels : la sortie anticipée (`return` , `break` , exception) fait remonter CS **automatiquement**. La version manuelle oublie toujours un chemin de sortie un jour ou l'autre — et un CS resté bas bloque tout le bus SPI. Le `= delete` de la copie fait partie de la réponse attendue.

Exercice 4 — FSM feu tricolore en classe

Énoncé. Transformer la FSM du TD 01 en classe avec `tick(maintenant_ms)` .

Corrigé

```
#include <cstdint>

class FeuTricolore {
public:
    enum class Etat : std::uint8_t { Vert, Orange, Rouge };

    explicit FeuTricolore(std::uint32_t maintenant)
        : etat_(Etat::Rouge), t_entree_(maintenant) {}

    void appuiPieton() { demande_pieton_ = true; }
    Etat etat() const { return etat_; }

    void tick(std::uint32_t maintenant) {
        const std::uint32_t ecoule = maintenant - t_entree_;
        switch (etat_) {
            case Etat::Vert:
                if (ecoule >= DUREE_VERT ||
                    (demande_pieton_ && ecoule >= VERT_MINI))
                    transition(Etat::Orange, maintenant);
                break;
            case Etat::Orange:
                if (ecoule >= DUREE_ORANGE) {
                    demande_pieton_ = false;           // demande servie
                    transition(Etat::Rouge, maintenant);
                }
                break;
            case Etat::Rouge:
                if (ecoule >= DUREE_ROUGE)
                    transition(Etat::Vert, maintenant);
                break;
        }
    }

private:
    void transition(Etat suivant, std::uint32_t maintenant) {
        etat_ = suivant;
        t_entree_ = maintenant;           // UN SEUL endroit où l'on change d'état
    }

    static constexpr std::uint32_t DUREE_VERT    = 5000;
    static constexpr std::uint32_t DUREE_ORANGE  = 1000;
    static constexpr std::uint32_t DUREE_ROUGE   = 5000;
    static constexpr std::uint32_t VERT_MINI     = 1000;

    Etat          etat_;
    std::uint32_t t_entree_;
    bool          demande_pieton_ = false;
};
```

Gains par rapport à la version C : - L'état et son horodatage sont **encapsulés** : personne ne peut mettre la FSM dans un état incohérent de l'extérieur (en C, n'importe qui pouvait écrire `f.etat = VERT;` sans

réinitialiser le chrono). - `enum class` : `Etat::Vert` ne se convertit pas silencieusement en entier et ne pollue pas l'espace de noms. - Le constructeur garantit un départ valide ; `transition()` centralise le changement d'état — un seul endroit à instrumenter pour tracer/déboguer. - On peut instancier **plusieurs feux indépendants** (`FeuTricolore nord, sud;`) — la version C à variables globales ne le permettait pas.

Exercice 5 (complément) — Lecture critique

Énoncé. Ce code compile. Donne trois critiques d'un point de vue « C++ embarqué » et propose les corrections.

```
class Journal {
public:
    Journal() { buffer = new char[4096]; }
    ~Journal() { }
    void log(std::string msg) { lignes.push_back(msg); }
private:
    char* buffer;
    std::vector<std::string> lignes;
};
```

Corrigé

1. **Fuite mémoire** : `new[]` dans le constructeur, jamais de `delete[]` dans le destructeur. Et si on ajoute le `delete[]`, la classe devient dangereuse à copier (double libération) → il faudrait aussi gérer ou supprimer copie/affectation (« règle des trois »). Correction embarquée : `char buffer[4096];` en membre (statique, pas de gestion) ou `std::unique_ptr<char[]>` sur gros système.
2. **`std::string` passé par valeur** : copie (et allocation) à chaque appel. Correction : `const std::string&`, ou mieux en embarqué `const char*` / `std::string_view`.
3. **`std::vector<std::string>` qui grossit sans borne** : allocations dynamiques répétées, fragmentation, et croissance illimitée sur un système qui tourne des mois → un jour, plus de RAM. Correction : journal **circulaire** de taille fixe (notre `TamponCirculaire` d'entrées de taille bornée), politique d'écrasement des plus anciens documentée.

Auto-évaluation avant la suite

Sans notes : expliquer RAI avec un exemple matériel ; justifier destructeur virtuel + `= delete` de la copie ; différence polymorphisme dynamique (vtable) / statique (template) et leurs coûts ; citer 3 éléments de la STL utilisables sur micro et 3 à éviter, avec la raison.

FORMATION EMBARQUÉE

TD 03 Arduino

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 03 — Arduino : énoncés et corrigés détaillés

Tous les corrigés sont testables sur <https://wokwi.com> sans matériel. Aucun corrigé n'utilise `delay()` dans la boucle : c'est le critère n°1.

Exercice 1 — Chenillard 4 LED, vitesse au potentiomètre, sans

`delay()`

Corrigé

```
const uint8_t LEDS[] = {8, 9, 10, 11};
const uint8_t NB_LEDS = sizeof LEDS / sizeof LEDS[0];
const uint8_t POT = A0;

uint8_t index_led = 0;
uint32_t derniere_avance = 0;

void setup() {
    for (uint8_t i = 0; i < NB_LEDS; i++) pinMode(LEDS[i], OUTPUT);
}

void loop() {
    // Période recalculée à CHAQUE tour : la vitesse réagit immédiatement.
    // map() : 0..1023 → 50..500 ms
    uint16_t periode = map(analogRead(POT), 0, 1023, 50, 500);

    uint32_t maintenant = millis();
    if (maintenant - derniere_avance >= periode) {
        derniere_avance = maintenant;

        digitalWrite(LEDS[index_led], LOW);           // éteindre l'actuelle
        index_led = (index_led + 1) % NB_LEDS;         // avancer (retour à 0 après 3)
        digitalWrite(LEDS[index_led], HIGH);           // allumer la suivante
    }
}
```

Points de correction : - Le tableau `LEDS[]` + `NB_LEDS` calculé par `sizeof` : ajouter une 5e LED = une seule ligne à changer. Quatre `digitalWrite` copiés-collés = zéro pointé en style. - `uint32_t` pour tout ce qui touche `millis()` (piège du `int` 16 bits). - La soustraction `maintenant - derniere_avance` survit au débordement de `millis()` (au bout de ~49 jours) — un `if (millis() > prochain_top)` non.

Exercice 2 — Compteur de passages (ultrason + OLED + bouton

Corrigé (structure complète)

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 oled(128, 64, &Wire, -1);

const uint8_t TRIG = 3, ECHO = 4, BTN_RAZ = 2;
const uint16_t SEUIL_CM = 50;           // "passage" = obstacle à moins de 50 cm

uint16_t compteur = 0;
bool      objet_present = false;        // pour ne compter qu'UNE fois par passage
uint32_t derniere_mesure = 0, derniere_maj_oled = 0;

// -- anti-rebond bouton (scrutation, pas d'interruption nécessaire ici)
bool      btn_stable = false, btn_brut_prec = false;
uint32_t btn_t_chgt = 0;

float mesurer_cm() {
    digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
    uint32_t duree = pulseIn(ECHO, HIGH, 30000UL); // timeout 30 ms : JAMAIS sans
    return (duree == 0) ? 999.0f : duree / 58.0f;   // 0 = timeout → "loin"
}

bool bouton_appuye(uint32_t maintenant) {
    bool brut = (digitalRead(BTN_RAZ) == LOW);
    if (brut != btn_brut_prec) { btn_t_chgt = maintenant; btn_brut_prec = brut; }
    if (maintenant - btn_t_chgt > 20 && brut != btn_stable) {
        btn_stable = brut;
        return btn_stable;           // true seulement sur le FRONT d'appui
    }
    return false;
}

void setup() {
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    pinMode(BTN_RAZ, INPUT_PULLUP);
    oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(3);
}

void loop() {
    uint32_t maintenant = millis();

    // 1) Mesure toutes les 60 ms (l'HC-SR04 n'aime pas être mitraillé)
    if (maintenant - derniere_mesure >= 60) {
        derniere_mesure = maintenant;
        bool proche = (mesurer_cm() < SEUIL_CM);
        if (proche && !objet_present) compteur++; // front d'ENTRÉE seulement
        objet_present = proche;
    }
}
```

```

}

// 2) Bouton de remise à zéro
if (bouton_appuye(maintenant)) compteur = 0;

// 3) OLED rafraîchi 5 fois/s (le rafraîchir à chaque tour ralentit tout)
if (maintenant - derniere_maj_oled >= 200) {
    derniere_maj_oled = maintenant;
    oled.clearDisplay();
    oled.setCursor(10, 20);
    oled.print(compteur);
    oled.display();
}
}

```

Points de correction : - **Détection de front** (`proche && !objet_present`) : sans elle, un objet immobile devant le capteur incrémente en continu. - **Timeout sur** `pulseIn` : sans lui, pas d'écho = boucle figée 1 s. - Trois activités (mesure, bouton, affichage) à trois cadences différentes dans une seule boucle — c'est exactement l'architecture attendue.

Exercice 3 — Thermostat à hystérésis en machine d'états

Corrigé

```
#include <DHT.h>

DHT dht(2, DHT22);
const uint8_t RELAIS = 7, POT_CONSIGNE = A0;
const float HYSTERESIS = 0.5f;

enum class Etat : uint8_t { Repos, Chauffe, Default };
Etat etat = Etat::Repos;

float consigne = 20.0f, temperature = NAN;
uint8_t echecs_lecture = 0;
uint32_t derniere_lecture = 0;

void setup() {
    pinMode(RELAIS, OUTPUT);
    digitalWrite(RELAIS, LOW);          // état SÛR au démarrage
    dht.begin();
    Serial.begin(115200);
}

void loop() {
    uint32_t maintenant = millis();

    // Consigne : 10..30 °C selon le potentiomètre
    consigne = 10.0f + analogRead(POT_CONSIGNE) * (20.0f / 1023.0f);

    // Le DHT22 se lit au plus toutes les 2 s
    if (maintenant - derniere_lecture >= 2000) {
        derniere_lecture = maintenant;
        float t = dht.readTemperature();
        if (isnan(t)) {
            if (++echecs_lecture >= 3) etat = Etat::Default;    // 3 échecs = panne
        } else {
            echecs_lecture = 0;
            temperature = t;
        }
    }

    switch (etat) {
        case Etat::Repos:
            digitalWrite(RELAIS, LOW);
            // On enclenche SOUS consigne - hystérésis (pas à la consigne pile)
            if (!isnan(temperature) && temperature < consigne - HYSTERESIS)
                etat = Etat::Chauffe;
            break;

        case Etat::Chauffe:
            digitalWrite(RELAIS, HIGH);
            // On coupe AU-DESSUS de consigne + hystérésis
            if (isnan(temperature) || temperature > consigne + HYSTERESIS)
```

```

        etat = Etat::Repos;
    break;

    case Etat::Default:
        digitalWrite(RELAIS, LOW);          // panne capteur → chauffage COUPÉ
        // Sortie de défaut uniquement si le capteur répond à nouveau
        if (echecs_lecture == 0 && !isnan(temperature)) etat = Etat::Repos;
        break;
    }
}

```

Pourquoi l'hystérésis : sans elle, autour de la consigne le relais « bat » (clignote) à chaque dixième de degré — usure mécanique et bruit. La bande morte de $\pm 0,5$ °C impose un cycle lent : c'est le comportement de tous les thermostats réels (et on le retrouvera à l'identique en SCL au TD 07).

Le point le plus noté : l'état `Default`. Un thermostat qui garde le chauffage allumé quand son capteur meurt est un danger. Réflexe : *que fait mon système quand chaque entrée tombe en panne ?* — question à se poser dans CHAQUE projet, jusqu'aux automates (modules 07-08).

Exercice 4 (complément) — Lecture de code : trouver les 5 défauts

Énoncé. Ce sketch « marche presque ». Liste ses défauts.

```

int compteur = 0;

void setup() {
    attachInterrupt(digitalPinToInterrupt(2), tic, FALLING);
    Serial.begin(9600);
}

void tic() {
    compteur++;
    Serial.println("tic!");
}

void loop() {
    delay(1000);
    Serial.println(compteur);
    if (compteur == 60) compteur = 0;
}

```

Corrigé

1. `Serial.println` dans l'ISR : les fonctions série utilisent des interruptions et des tampons — appelées depuis une ISR, elles peuvent bloquer définitivement. Une ISR pose un drapeau, point.
2. `compteur` non `volatile` : partagé ISR/boucle, le compilateur peut en garder une copie en registre dans `loop()`.
3. **Lecture non atomique** : `compteur` est un `int` 16 bits sur Uno — sa lecture dans `loop()` peut être coupée par l'ISR. Il faut `noInterrupts()/interrupts()` autour de la lecture (et de la remise à 0).

4. `if (compteur == 60)` : l'ISR peut faire passer le compteur de 59 à 61 entre deux tests (deux fronts rapides) → la condition n'est jamais vraie. Écrire `>= 60`.
 5. `pinMode(2, INPUT_PULLUP)` **absent** : broche flottante → fronts parasites en rafale. (Bonus : aucun anti-rebond si la source est un bouton mécanique.)
-

Auto-évaluation avant le TP 1

Sans notes : écrire le motif `millis()` de tête ; expliquer front vs niveau ; justifier l'hystérésis ; citer les règles d'une ISR ; expliquer pourquoi trois cadences différentes peuvent cohabiter dans une seule `loop()`.

➡ Passe au [TP 1 — Station météo](#).

FORMATION EMBARQUÉE

TD 04 Vhdl

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 04 — VHDL : énoncés et corrigés détaillés

Tous les corrigés se simulent avec GHDL + GTKWave (ou edaplayground.com). Rappel de la discipline : **rien ne part en synthèse sans testbench.**

Exercice 1 — Multiplexeur 4 vers 1

Énoncé. Mux 4 → 1 (2 bits de sélection), en version concurrente puis en version process.

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;

entity mux4 is
    port (
        e0, e1, e2, e3 : in  std_logic;
        sel             : in  std_logic_vector(1 downto 0);
        s               : out std_logic
    );
end entity;

-- Version 1 : affectation sélectionnée (concurrente)
architecture concurrente of mux4 is
begin
    with sel select
        s <= e0 when "00",
            e1 when "01",
            e2 when "10",
            e3 when others; -- "others" couvre "11" ET les états 'X', 'U'...
end architecture;

-- Version 2 : process combinatoire
architecture avec_process of mux4 is
begin
    process (e0, e1, e2, e3, sel) -- TOUTES les entrées lues sont dans la liste
    begin
        case sel is
            when "00" => s <= e0;
            when "01" => s <= e1;
            when "10" => s <= e2;
            when others => s <= e3; -- chaque chemin affecte s → pas de latch
        end case;
    end process;
end architecture;
```

Points de correction : - `when others` est **obligatoire** avec `std_logic_vector` : le type a 9 valeurs possibles par bit (pas seulement 0/1), le `case` doit être exhaustif. - Liste de sensibilité complète + `s`

affecté dans toutes les branches : les deux conditions pour que la version process décrive bien du combinatoire pur (aucune bascule, aucun latch). - Les deux architectures se synthétisent **en exactement le même circuit**.

Testbench (exhaustif — 4 entrées, on peut tout tester) :

```
entity tb_mux4 is end entity;

architecture sim of tb_mux4 is
    signal e0, e1, e2, e3, s : std_logic;
    signal sel : std_logic_vector(1 downto 0);
begin
    dut : entity work.mux4(concurrente)
        port map (e0, e1, e2, e3, sel, s);

    stim : process
    begin
        (e0, e1, e2, e3) <= std_logic_vector("1010");
        sel <= "00"; wait for 10 ns;
        assert s = '1' report "sel=00 : attendu e0=1" severity error;
        sel <= "01"; wait for 10 ns;
        assert s = '0' report "sel=01 : attendu e1=0" severity error;
        sel <= "10"; wait for 10 ns;
        assert s = '1' report "sel=10 : attendu e2=1" severity error;
        sel <= "11"; wait for 10 ns;
        assert s = '0' report "sel=11 : attendu e3=0" severity error;
        report "mux4 : tous les tests passent";
        wait;
    end process;
end architecture;
```


Exercice 2 — Compteur BCD 0-9 + décodeur 7 segments

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity compteur_bcd is
    port (
        clk, reset, en : in std_logic;
        chiffre         : out unsigned(3 downto 0);
        seg             : out std_logic_vector(6 downto 0) -- gfedcba, actif bas
    );
end entity;

architecture rtl of compteur_bcd is
    signal cpt : unsigned(3 downto 0) := (others => '0');
begin
    -- Partie SÉQUENTIELLE : le compteur (des bascules)
    process (clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                cpt <= (others => '0');
            elsif en = '1' then
                if cpt = 9 then -- retour à 0 après 9 : BCD, pas binaire !
                    cpt <= (others => '0');
                else
                    cpt <= cpt + 1;
                end if;
            end if;
        end if;
    end process;

    chiffre <= cpt;

    -- Partie COMBINATOIRE : le décodeur 7 segments (une simple table)
    with cpt select
        seg <= "1000000" when "0000", -- 0
              "1111001" when "0001", -- 1
              "0100100" when "0010", -- 2
              "0110000" when "0011", -- 3
              "0011001" when "0100", -- 4
              "0010010" when "0101", -- 5
              "0000010" when "0110", -- 6
              "1111000" when "0111", -- 7
              "0000000" when "1000", -- 8
              "0010000" when "1001", -- 9
              "1111111" when others; -- éteint si valeur invalide
    end architecture;
```

Extrait de testbench avec horloge générée et vérification du retour à zéro :

```

architecture sim of tb_compteur_bcd is
    signal clk, reset, en : std_logic := '0';
    signal chiffre : unsigned(3 downto 0);
    signal seg : std_logic_vector(6 downto 0);
    constant PERIODE : time := 10 ns;
begin
    dut : entity work.compteur_bcd port map (clk, reset, en, chiffre, seg);

    clk <= not clk after PERIODE / 2;    -- horloge perpétuelle en une ligne

    stim : process
    begin
        reset <= '1'; wait for 2 * PERIODE;
        reset <= '0'; en <= '1';
        for i in 0 to 9 loop
            assert chiffre = i
                report "attendu " & integer'image(i) severity error;
            wait for PERIODE;
        end loop;
        wait for PERIODE / 4;            -- s'écarter du front avant de lire
        assert chiffre = 0 report "pas revenu a 0 apres 9 !" severity error;
        report "compteur_bcd : OK";
        wait;
    end process;
end architecture;

```

Points de correction : séparation nette séquentiel (compteur) / combinatoire (décodeur) ; le test du passage 9 → 0 est LE cas intéressant — un testbench qui ne teste que 0 → 3 ne vaut rien.

Exercice 3 — Anti-rebond matériel (synchroniseur + filtre 20 ms)

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity debounce is
  generic (
    F_CLK      : natural := 50_000_000;
    DUREE_MS   : natural := 20
  );
  port (
    clk        : in  std_logic;
    btn_brut   : in  std_logic;    -- asynchrone, rebondissant
    btn_net    : out std_logic;    -- propre, synchrone
    front      : out std_logic     -- '1' pendant 1 cycle sur front montant net
  );
end entity;

architecture rtl of debounce is
  constant CYCLES_STABLES : natural := F_CLK / 1000 * DUREE_MS;
  signal sync0, sync1 : std_logic := '0';    -- synchroniseur 2 étages
  signal etat_net      : std_logic := '0';
  signal etat_prec     : std_logic := '0';
  signal cpt           : natural range 0 to CYCLES_STABLES := 0;
begin
  process (clk)
  begin
    if rising_edge(clk) then
      -- 1) Synchronisation : JAMAIS d'entrée asynchrone directe
      sync0 <= btn_brut;
      sync1 <= sync0;

      -- 2) Filtre : l'état doit rester stable CYCLES_STABLES cycles
      if sync1 /= etat_net then
        if cpt = CYCLES_STABLES - 1 then
          etat_net <= sync1;    -- état confirmé
          cpt <= 0;
        else
          cpt <= cpt + 1;
        end if;
      else
        cpt <= 0;              -- retombé pareil : on repart de zéro
      end if;

      -- 3) Mémoire pour le détecteur de front
      etat_prec <= etat_net;
    end if;
  end process;

  btn_net <= etat_net;
```

```
front    <= etat_net and not etat_prec;    -- exactement 1 cycle  
end architecture;
```

Points de correction : - Les **2 bascules de synchronisation** d'abord : sans elles, la métastabilité peut corrompre tout le circuit aval. C'est non négociable pour toute entrée externe. - Le compteur repart de zéro à **chaque** instabilité : 20 ms *consécutives* sont exigées. - Astuce testbench : passer `DUREE_MS => 1` et `F_CLK => 1000` en `generic map` pour simuler vite — c'est exactement à ça que servent les generics.

Exercice 4 — Feu tricolore avec bouton piéton

Corrigé (structure — réutilise le pattern du cours §4.3)

```
architecture rtl of feu_pieton is
    type t_etat is (VERT, ORANGE, ROUGE);
    signal etat : t_etat := ROUGE;
    signal cpt : unsigned(27 downto 0) := (others => '0');
    signal demande : std_logic := '0';
begin
    process (clk)
    begin
        if rising_edge(clk) then
            -- Mémorisation de la demande (front déjà nettoyé par debounce)
            if btn_front = '1' then
                demande <= '1';
            end if;

            cpt <= cpt + 1;
            case etat is
                when VERT =>
                    -- fin normale OU demande piéton après le délai minimal
                    if cpt = DUREE_VERT
                        or (demande = '1' and cpt >= DUREE_VERT_MINI) then
                        etat <= ORANGE;
                        cpt <= (others => '0');
                    end if;
                when ORANGE =>
                    if cpt = DUREE_ORANGE then
                        etat <= ROUGE;
                        cpt <= (others => '0');
                        demande <= '0'; -- demande servie
                    end if;
                when ROUGE =>
                    if cpt = DUREE_ROUGE then
                        etat <= VERT;
                        cpt <= (others => '0');
                    end if;
            end case;
        end if;
    end process;

    feu_vert <= '1' when etat = VERT else '0';
    feu_orange <= '1' when etat = ORANGE else '0';
    feu_rouge <= '1' when etat = ROUGE else '0';
end architecture;
```

À remarquer : c'est le même algorithme que le TD 01 en C et le TD 02 en C++ — drapeau mémorisé, durée minimale de vert, remise à zéro à la desserte. Seul le « moteur » change : ici, l'évaluation a lieu à chaque front d'horloge, en matériel. Si tu as vu cette correspondance tout seul, le paradigme est acquis.

Exercice 5 — Émetteur UART 115200 bauds

Corrigé complet

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity uart_tx is
    generic (
        F_CLK : natural := 50_000_000;
        BAUD  : natural := 115_200
    );
    port (
        clk      : in  std_logic;
        tx_start : in  std_logic;           -- impulsion 1 cycle
        tx_data  : in  std_logic_vector(7 downto 0);
        tx       : out std_logic;           -- la ligne série
        busy     : out std_logic
    );
end entity;

architecture rtl of uart_tx is
    constant TICKS_PAR_BIT : natural := F_CLK / BAUD; -- 50e6/115200 ≈ 434
    type t_etat is (REPOS, START, DONNEES, STOP);
    signal etat      : t_etat := REPOS;
    signal cpt_tick  : natural range 0 to TICKS_PAR_BIT - 1 := 0;
    signal idx_bit   : natural range 0 to 7 := 0;
    signal registre  : std_logic_vector(7 downto 0) := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            case etat is
                when REPOS =>
                    tx      <= '1';           -- ligne au repos = état haut
                    busy    <= '0';
                    if tx_start = '1' then
                        registre <= tx_data;   -- CAPTURER la donnée maintenant :
                        cpt_tick <= 0;          -- l'appelant peut la changer après
                        busy     <= '1';
                        etat     <= START;
                    end if;

                when START =>
                    tx <= '0';               -- bit de start
                    if cpt_tick = TICKS_PAR_BIT - 1 then
                        cpt_tick <= 0;
                        idx_bit  <= 0;
                        etat     <= DONNEES;
                    else
                        cpt_tick <= cpt_tick + 1;
                    end if;
            end case;
        end if;
    end process;
end architecture;
```

```

when DONNEES =>
    tx <= registre(idx_bit);    -- LSB en premier (norme UART)
    if cpt_tick = TICKS_PAR_BIT - 1 then
        cpt_tick <= 0;
        if idx_bit = 7 then
            etat <= STOP;
        else
            idx_bit <= idx_bit + 1;
        end if;
    else
        cpt_tick <= cpt_tick + 1;
    end if;

when STOP =>
    tx <= '1';                -- bit de stop
    if cpt_tick = TICKS_PAR_BIT - 1 then
        etat <= REPOS;
    else
        cpt_tick <= cpt_tick + 1;
    end if;

end case;
end if;
end process;
end architecture;

```

Testbench (envoi de 0x55 = **01010101** , motif idéal : il alterne) :

```

architecture sim of tb_uart_tx is
    signal clk, tx_start, tx, busy : std_logic := '0';
    signal tx_data : std_logic_vector(7 downto 0);
    -- Astuce : generics réduits pour une simulation courte et lisible
    constant F : natural := 10 * 115200;    -- 10 ticks par bit seulement
begin
    dut : entity work.udp_tx
        generic map (F_CLK => F, BAUD => 115_200)
        port map (clk, tx_start, tx_data, tx, busy);

    clk <= not clk after 50 ns;

    stim : process
    begin
        wait for 200 ns;
        tx_data <= x"55";
        tx_start <= '1'; wait for 100 ns; tx_start <= '0';    -- impulsion
        wait until busy = '0';                                -- fin d'émission
        report "trame emise, verifier le chronogramme dans GTKWave";
        wait;
    end process;
end architecture;

```

Ce qu'on vérifie au chronogramme : ligne haute au repos → start bas pendant 1 temps bit →

1,0,1,0,1,0,1,0 (0x55, LSB d'abord) → stop haut, chaque bit durant exactement **TICKS_PAR_BIT** cycles.

Points de correction : la capture de **tx_data** dans **registre** à l'acceptation (l'oublier = donnée

corrompue si l'appelant la change en cours d'émission), le LSB en premier, et `busy` qui couvre toute la trame.

Auto-évaluation avant le TP 2

Sans notes : écrire un process synchrone canonique ; expliquer latch involontaire et comment l'éviter ; justifier le synchroniseur 2 bascules ; dessiner la trame UART 8N1 ; expliquer pourquoi `wait for` est interdit en synthèse.

 Passe au [TP 2 — UART complet en VHDL](#).

FORMATION EMBARQUÉE

TD 05 Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 05 — Java : énoncés et corrigés détaillés

Corrigés compilables avec un JDK ≥ 17 (`javac` puis `java`). Chaque exercice illustre une brique du rôle de Java autour de l'embarqué : modélisation objet, concurrence, décodage binaire, traitement de données.

Exercice 1 — Hiérarchie de capteurs et journalisation CSV

Énoncé. `Capteur` (abstraite), `CapteurTemp`, `CapteurHum`, et une classe `Station` qui interroge une `List<Capteur>` et journalise en CSV.

Corrigé

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.time.Instant;
import java.util.List;

abstract class Capteur {
    private final String nom;
    private final String unite;

    protected Capteur(String nom, String unite) {
        this.nom = nom;
        this.unite = unite;
    }

    public String getNom() { return nom; }
    public String getUnite() { return unite; }

    /** Chaque capteur concret fournit SA façon de mesurer. */
    public abstract double lire();
}

class CapteurTemp extends Capteur {
    CapteurTemp() { super("temperature", "°C"); }
    @Override public double lire() { return 18 + Math.random() * 10; } // simulé
}

class CapteurHum extends Capteur {
    CapteurHum() { super("humidite", "%"); }
    @Override public double lire() { return 30 + Math.random() * 40; }
}

class Station {
    private final List<Capteur> capteurs;
    private final Path fichier;

    Station(List<Capteur> capteurs, Path fichier) {
        this.capteurs = List.copyOf(capteurs); // copie défensive : la liste
        this.fichier = fichier; // de l'appelant peut changer
    }

    /** Une ligne CSV par capteur : horodatage;nom;valeur;unite */
    public void releve() throws IOException {
        try (PrintWriter out = new PrintWriter(Files.newBufferedWriter(
            fichier, StandardOpenOption.CREATE, StandardOpenOption.APPEND))) {
            Instant t = Instant.now();
            for (Capteur c : capteurs) {
                out.printf("%s;%s;%.2f;%s\n", t, c.getNom(), c.lire(), c.getUnite());
            }
        } // try-with-resources : fichier fermé même en cas d'exception
    }
}
```

```

    }
}

public class Main {
    public static void main(String[] args) throws IOException {
        Station station = new Station(
            List.of(new CapteurTemp(), new CapteurHum()),
            Path.of("mesures.csv"));
        for (int i = 0; i < 5; i++) station.releve();
        System.out.println("5 relevés écrits dans mesures.csv");
    }
}

```

Points de correction : le polymorphisme (`Station` ne connaît que `Capteur` , ajouter un capteur = zéro modification de `Station`) ; la copie défensive `List.copyOf` ; le try-with-resources (le RAII de Java, cf. TD 02 exercice 3 — même idée, autre langage).

Exercice 2 — Producteur/consommateur avec `BlockingQueue`

Corrigé

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.TimeUnit;

record Mesure(String capteur, double valeur, long horodatageMs) {}

public class ProdCons {
    public static void main(String[] args) throws InterruptedException {
        // File BORNÉE (64) : si le consommateur décroche, le producteur
        // se bloque au lieu de remplir la RAM – même logique que le ring
        // buffer plein du TD 01.
        BlockingQueue<Mesure> file = new ArrayBlockingQueue<>(64);

        Thread producteur = new Thread(() -> {
            try {
                while (!Thread.currentThread().isInterrupted()) {
                    file.put(new Mesure("temp", 18 + Math.random() * 10,
                                          System.currentTimeMillis()));
                    Thread.sleep(200); // 5 mesures/s
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); // on RESTAURE le drapeau
            }
        }, "acquisition");

        Thread consommateur = new Thread(() -> {
            try {
                while (!Thread.currentThread().isInterrupted()) {
                    // poll avec timeout plutôt que take() : permet de tester
                    // régulièrement le drapeau d'interruption
                    Mesure m = file.poll(500, TimeUnit.MILLISECONDS);
                    if (m != null)
                        System.out.printf("traite : %s = %.1f%n", m.capteur(), m.valeur());
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "traitement");

        producteur.start();
        consommateur.start();

        Thread.sleep(3000); // le programme tourne 3 s...
        producteur.interrupt(); // ...puis arrêt PROPRE des deux threads
        consommateur.interrupt();
        producteur.join();
        consommateur.join();
        System.out.println("arret propre, file restante = " + file.size());
    }
}
```

```
}  
}
```

Points de correction : - `BlockingQueue` remplace mutex + condition + tampon écrits à la main : toute la synchronisation est dans la structure. - **Arrêt propre :** `interrupt()` + `join()` , et le `catch` qui restaure le drapeau. Un thread tué brutalement (il n'y a d'ailleurs pas de moyen sûr de le faire en Java) laisse des ressources en vrac — même souci qu'un firmware coupé en pleine écriture flash. - La file **bornée** est un choix de conception, pas un détail (contre- pression / backpressure).

Exercice 3 — Décodeur de trame binaire (le piège des `byte` signés)

Énoncé. `byte[] {0xA5, id, hi, lo}` → objet `Mesure` , avec validation.

Corrigé

```
import java.util.Optional;

record TrameMesure(int id, int valeur) {}

class DecodeurTrame {
    static final int TETE = 0xA5;
    static final int LONGUEUR = 4;

    /** Optional plutôt que null ou exception : trame invalide = cas NORMAL
     *  sur un bus réel, pas un cas exceptionnel. */
    static Optional<TrameMesure> decoder(byte[] trame) {
        if (trame == null || trame.length != LONGUEUR)
            return Optional.empty();

        // PIÈGE CENTRAL : en Java, byte est SIGNÉ (-128..127).
        // (byte)0xA5 vaut -91 ! Le masque & 0xFF "remonte" en int non signé.
        if ((trame[0] & 0xFF) != TETE)
            return Optional.empty();

        int id = trame[1] & 0xFF;
        int valeur = ((trame[2] & 0xFF) << 8) | (trame[3] & 0xFF);
        return Optional.of(new TrameMesure(id, valeur));
    }
}

public class TestDecodeur {
    public static void main(String[] args) {
        byte[] ok = {(byte) 0xA5, 0x07, (byte) 0x01, (byte) 0xF4}; // valeur 500
        byte[] ko = {(byte) 0x55, 0x07, 0x00, 0x00}; // mauvaise tête

        System.out.println(DecodeurTrame.decoder(ok)); // Optional[TrameMesure[id=7, valeur=500]
        System.out.println(DecodeurTrame.decoder(ko)); // Optional.empty
        System.out.println(DecodeurTrame.decoder(new byte[2])); // Optional.empty
    }
}
```

LE point de correction : sans `& 0xFF`, `trame[0] != TETE` compare -91 à 165 → toute trame valide est **rejetée**. C'est le bug Java n°1 en manipulation binaire (sockets, ports série, fichiers). Compare avec le C (TD 01 ex. 4) : mêmes vérifications, mêmes décalages — seule la gestion du signe diffère.

Exercice 4 — Statistiques d'un CSV avec les streams

Énoncé. Lire un CSV `horodatage;capteur;valeur`, sortir min/max/moyenne par capteur.

Corrigé

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.DoubleSummaryStatistics;
import java.util.Map;
import java.util.stream.Collectors;

public class StatsCsv {
    public static void main(String[] args) throws IOException {
        Map<String, DoubleSummaryStatistics> stats;
        try (var lignes = Files.lines(Path.of("mesures.csv"))) {
            stats = lignes
                .map(l -> l.split(";"))
                .filter(c -> c.length >= 3) // ligne malformée → ignorée
                .collect(Collectors.groupingBy(
                    c -> c[1], // clé : nom du capteur
                    Collectors.summarizingDouble(c -> {
                        try { return Double.parseDouble(c[2]); }
                        catch (NumberFormatException e) { return Double.NaN; }
                    }
                ));
        }

        stats.forEach((capteur, s) ->
            System.out.printf("%-12s : n=%d min=%.2f max=%.2f moy=%.2f%n",
                capteur, s.getCount(), s.getMin(), s.getMax(), s.getAverage()));
    }
}
```

Points de correction : `Files.lines` dans un try-with-resources (c'est un flux paresseux qui tient le fichier ouvert) ; les lignes malformées sont **filtrées, pas fatales** ; `DoubleSummaryStatistics` donne min/max/moyenne en une seule passe — pas trois parcours du fichier.

Exercice 5 (complément) — Question de conception

Énoncé. Ton ESP32 envoie une mesure JSON par seconde en MQTT. Tu dois : l'historiser, afficher la dernière valeur sur une page web, alerter au-delà d'un seuil. Découpe une application Java en classes/threads et justifie.

Corrigé (éléments attendus)

- **Un thread d'ingestion** : le callback MQTT ne fait que désérialiser et poser la mesure dans une `BlockingQueue` (règle « ISR courte », version serveur : un callback de bibliothèque réseau ne doit jamais bloquer).
- **Un thread de traitement** : consomme la file → écrit en base (SQLite/ PostgreSQL), met à jour un `volatile Mesure derniere` (ou `AtomicReference`), évalue le seuil.
- **Alerte avec hystérésis et anti-rafale** : même logique qu'au TD 03 — on n'envoie pas 60 e-mails/minute quand la valeur oscille autour du seuil.

- **Serveur HTTP** dans son propre pool de threads, qui ne lit que des données déjà prêtes (jamais la base sur le chemin chaud).
 - Séparation en couches : **transport** (MQTT), **domaine** (Mesure, règles d'alerte — **testables sans réseau**), **persistance**, **web**. La logique métier ne dépend d'aucune bibliothèque : c'est le même principe que l'interface **IAfficheur** du TD 02.
-

Auto-évaluation

Sans notes : expliquer `& 0xFF` sur un `byte` ; try-with-resources vs RAII C++ ; pourquoi une file bornée ; le protocole d'arrêt propre d'un thread ; et citer deux endroits où Java touche l'industriel (Modbus TCP, MQTT, Android, SCADA type Ignition).

FORMATION EMBARQUÉE

TD 06 Autres Langages

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 06 — Autres langages/outils : énoncés et corrigés

Exercice 1 — Lire l'assembleur produit par GCC

Énoncé. Compile cette fonction avec `-O0` puis `-O2`, désassemble, explique deux optimisations observées.

```
#include <stdint.h>
uint32_t somme(const uint32_t *t, uint32_t n) {
    uint32_t s = 0;
    for (uint32_t i = 0; i < n; i++)
        s += t[i] * 2;
    return s;
}
```

```
gcc -O0 -S somme.c -o somme_00.s
gcc -O2 -S somme.c -o somme_02.s
# ou, en croisé : arm-none-eabi-gcc -mcpu=cortex-m4 -O2 -S somme.c
# ou en ligne : https://godbolt.org (le plus confortable)
```

Corrigé (observations attendues)

1. **Variables en registres** : en `-O0`, `s` et `i` vivent **en pile** — chaque tour fait des chargements/rangements mémoire (`ldr` / `str` autour de chaque opération). En `-O2`, tout tient dans des registres : le corps de boucle se réduit à ~3 instructions. C'est la raison pour laquelle on ne mesure JAMAIS de performance en `-O0`.
2. **Multiplication remplacée** : `* 2` devient un décalage (`lsl #1`) ou s'intègre au mode d'adressage — le compilateur fait tout seul les « astuces » bit à bit du module 01.
3. (Bonus) En `-O2` sur x86, GCC **vectorise** souvent (instructions SIMD traitant 4 éléments par tour) et/ou déroule la boucle ; sur Cortex-M, il fusionne le test de fin avec la soustraction.
4. (Bonus, à retenir) : si `s` était `volatile`, quasi toutes ces optimisations disparaîtraient — relie ça au rôle exact de `volatile`.

Exercice 2 — Makefile pour un projet 3 fichiers

Énoncé. `main.c`, `capteur.c`, `affichage.c` (+ leurs `.h`) : cibles `all`, `clean`, reconstruction minimale.

Corrigé

```
CC      := gcc
CFLAGS  := -Wall -Wextra -O2 -MMD -MP
SRCS    := main.c capteur.c affichage.c
OBJS    := $(SRCS:.c=.o)
DEPS    := $(OBJS:.o=.d)
CIBLE   := app

all: $(CIBLE)

$(CIBLE): $(OBJS)
    $(CC) $(CFLAGS) $^ -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(DEPS) $(CIBLE)

-include $(DEPS)

.PHONY: all clean
```

Points de correction : - La **reconstruction minimale** vient de deux mécanismes : la règle générique `%.o: %.c` (make compare les dates `.c/.o`) et les fichiers de dépendances générés par `-MMD -MP` puis inclus par `-include $(DEPS)` — sans eux, modifier `capteur.h` ne recompile pas `main.c` qui l'inclut : c'est LE piège classique. - `.PHONY : all` et `clean` ne sont pas des fichiers ; sans cette ligne, un fichier nommé `clean` casserait la cible. - `$@` = la cible, `$<` = la première dépendance, `$^` = toutes. - Test : `make` (tout compile) → `touch capteur.h` → `make` (main.o et capteur.o se recompilent, pas affichage.o) → `make` (rien à faire).

Exercice 3 — Deux tâches FreeRTOS sur ESP32

Énoncé. Une tâche clignote une LED à 2 Hz, l'autre imprime un compteur chaque seconde ; communication par queue.

Corrigé (IDE Arduino, cible ESP32 — FreeRTOS est déjà là)

```
#include <Arduino.h>

QueueHandle_t file_compteur;

void tacheBlink(void *param) {
    pinMode(2, OUTPUT);
    for (;;) {
        digitalWrite(2, !digitalRead(2));
        // vTaskDelay REND le CPU : l'autre tâche (et le Wi-Fi !) tournent.
        // Un delay() Arduino ferait pareil ici, mais prends l'habitude RTOS.
        vTaskDelay(pdMS_TO_TICKS(250));    // 250 ms → 2 Hz
    }
}

void tacheCompteur(void *param) {
    uint32_t n = 0;
    for (;;) {
        n++;
        xQueueSend(file_compteur, &n, 0);    // 0 : n'attend pas si pleine
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void setup() {
    Serial.begin(115200);
    file_compteur = xQueueCreate(8, sizeof(uint32_t));

    // (fonction, nom, pile en MOTS, param, priorité, handle, cœur)
    xTaskCreatePinnedToCore(tacheBlink, "blink", 2048, NULL, 1, NULL, 1);
    xTaskCreatePinnedToCore(tacheCompteur, "cpt", 2048, NULL, 1, NULL, 1);
}

void loop() {
    uint32_t n;
    // loop() sert de tâche "affichage" : elle BLOQUE sur la queue
    if (xQueueReceive(file_compteur, &n, portMAX_DELAY) == pdTRUE)
        Serial.printf("compteur = %lu\n", (unsigned long) n);
}
```

Points de correction : - Le compteur passe par **la queue**, pas par une variable globale : pas de **volatile**, pas de section critique, transfert par copie — c'est tout l'intérêt. - **vTaskDelay** (bloquant-coopératif) et non une boucle d'attente active qui affamerait les tâches de priorité inférieure. - Taille de pile : 2048 est confortable ; en cas de plantages aléatoires, premier réflexe RTOS = **uxTaskGetStackHighWaterMark()**.

Exercice 4 — Station météo en MicroPython (comparaison)

Corrigé (Pico + DHT22 + affichage console, complet)

```
from machine import Pin
import dht, time

capteur = dht.DHT22(Pin(15))
led = Pin(25, Pin.OUT)

SEUIL_ALERTE = 28.0

while True:
    try:
        capteur.measure()
        t = capteur.temperature()
        h = capteur.humidity()
        print("T = {:.1f} C  H = {:.0f} %".format(t, h))
        led.value(1 if t > SEUIL_ALERTE else 0)
    except OSError:
        print("capteur muet, nouvelle tentative") # panne = cas géré, pas crash
    time.sleep(2)
```

Analyse attendue (le fond de l'exercice) : la version MicroPython tient en 15 lignes et se met au point en direct via le REPL — contre une heure et des bibliothèques en C++. En échange : ~100× plus lent, RAM consommée par l'interpréteur, latences avec le ramasse-miettes → parfait pour un prototype ou un capteur lent, disqualifié pour du temps réel serré. Conclusion à formuler : **le langage se choisit par les contraintes, pas par le confort.**

Auto-évaluation

Sans notes : expliquer `-O0` vs `-O2` et pourquoi `volatile` bride l'optimiseur ; écrire de tête la règle générique `%.o: %.c` avec `$< / $@` ; justifier une queue RTOS face à une globale partagée ; donner deux critères qui disqualifient MicroPython pour une application donnée.

FORMATION EMBARQUÉE

TD 07 Siemens

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 07 — Siemens TIA Portal : énoncés et corrigés détaillés

Les corrigés LADDER sont donnés en schéma ASCII + description réseau par réseau (à reproduire dans TIA Portal), les corrigés SCL en code complet. Tout se teste avec PLCSIM.

Exercice 1 — Va-et-vient de chariot avec arrêt d'urgence

Énoncé. Un chariot va et vient entre deux fins de course `fdc_gauche` (%I0.2) et `fdc_droite` (%I0.3). Départ par `bp_depart` (%I0.0). Arrêt d'urgence `arret_urgence` (%I0.1, **contact NF câblé** : 1 = tout va bien). Après un arrêt d'urgence, un réarmement par `bp_rearmement` (%I0.4) est obligatoire avant tout redémarrage. Sorties : `mvt_droite` (%Q0.0), `mvt_gauche` (%Q0.1).

Corrigé

Table des variables :

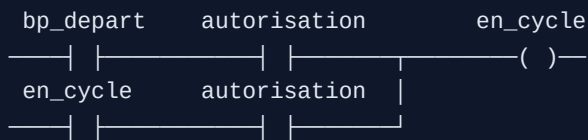
Nom	Type	Adresse	Commentaire
bp_depart	Bool	%I0.0	NO
arret_urgence	Bool	%I0.1	NF câblé : 1 = OK, 0 = AU appuyé
fdc_gauche / fdc_droite	Bool	%I0.2 / %I0.3	fins de course
bp_rearmement	Bool	%I0.4	NO
autorisation	Bool	%M0.0	mémoire de réarmement
en_cycle	Bool	%M0.1	cycle demandé
sens_droite	Bool	%M0.2	mémoire de sens
mvt_droite / mvt_gauche	Bool	%Q0.0 / %Q0.1	contacteurs

Réseau 1 — Autorisation (réarmement obligatoire) :

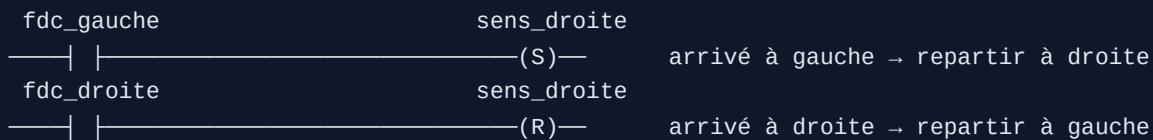
```
bp_rearmement  arret_urgence          autorisation
—| |—————| |—————( )—
autorisation  arret_urgence
—| |—————| |—————
```

`autorisation` s'établit par le réarmement (si l'AU est relâché) et se maintient **tant que** l'AU reste relâché : le moindre AU la fait retomber, et elle ne revient pas seule — il faut réappuyer sur réarmement.

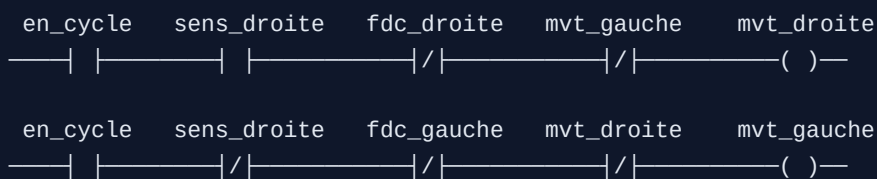
Réseau 2 — Demande de cycle (auto-maintien) :



Réseau 3 — Mémoire de sens (basculer S/R par les fins de course) :



Réseaux 4 et 5 — Sorties avec verrouillage mutuel :



Points de correction (les non-négociables) : 1. **AU en contact NF** : fil coupé = AU déclenché (sécurité positive). Un AU en NO qui perd son fil ne protégera plus jamais. 2. **Réarmement obligatoire** : l'AU relâché ne suffit pas à repartir (norme machine : pas de redémarrage automatique après coupure sécurité). 3. **Verrouillage mutuel** des deux sorties (**mvt_droite** interdit **mvt_gauche** et réciproquement) : deux contacteurs de sens fermés ensemble = court-circuit. 4. Chaque sortie est aussi coupée par **sa** fin de course.

Exercice 2 — FB « Moyenne_Glissante » en SCL

Énoncé. FB avec Array de 10 Real, entrée **nouvelle_mesure**, entrée **echantillonner** (à traiter sur front), sortie **moyenne**. L'instancier deux fois.

Corrigé

Interface du FB :

Section	Nom	Type
Input	nouvelle_mesure	Real
Input	echantillonner	Bool
Output	moyenne	Real
Static	mesures	Array[0..9] of Real
Static	index	Int
Static	nb_valides	Int
Static	echant_prec	Bool
Temp	i	Int
Temp	somme	Real

Corps en SCL :

```
// Détection de front montant sur la commande d'échantillonnage :
// un FB est appelé À CHAQUE CYCLE ; sans front, on stockerait la même
// mesure des centaines de fois par seconde.
IF #echantillonner AND NOT #echant_prec THEN
    #mesures[#index] := #nouvelle_mesure;
    #index := (#index + 1) MOD 10;           // tampon circulaire (cf. TD 01 !)
    IF #nb_valides < 10 THEN
        #nb_valides := #nb_valides + 1;     // démarrage : moyenne sur n < 10
    END_IF;
END_IF;
#echant_prec := #echantillonner;

// Moyenne sur les échantillons réellement présents
#somme := 0.0;
FOR #i := 0 TO #nb_valides - 1 DO
    #somme := #somme + #mesures[#i];
END_FOR;

IF #nb_valides > 0 THEN
    #moyenne := #somme / INT_TO_REAL(#nb_valides);
ELSE
    #moyenne := 0.0;                       // jamais de division par zéro
END_IF;
```

Double instanciation (dans OB1) : appeler le FB deux fois en créant deux **DB d'instance** distincts (**DB_Moyenne_Temp** , **DB_Moyenne_Pression**) — chaque instance garde SON tableau et SON index. C'est exactement « une classe, deux objets » (TD 02).

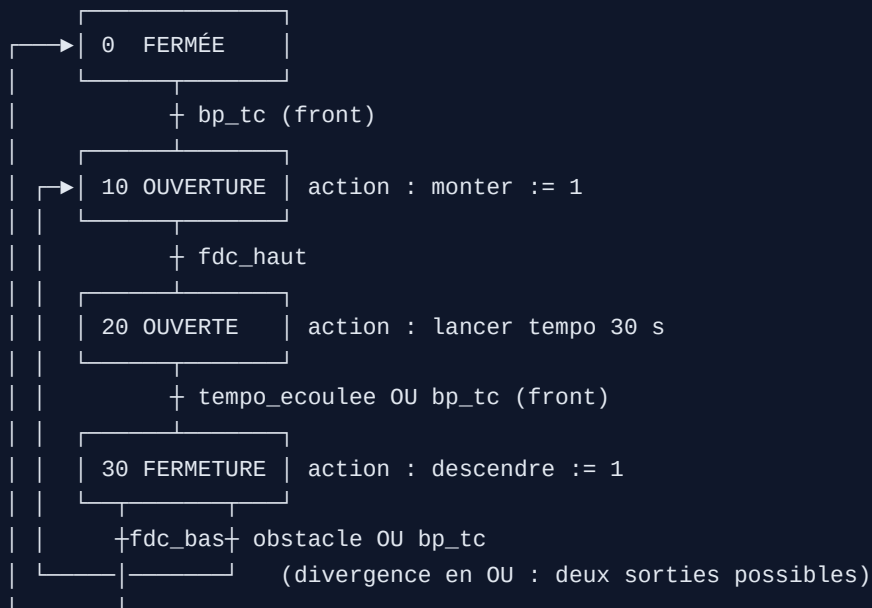
Points de correction : la détection de front interne (l'erreur n°1 des débutants en FB) ; le démarrage progressif (**nb_valides**) ; la garde de division ; la conversion explicite **INT_TO_REAL** (SCL ne convertit pas silencieusement).

Exercice 3 — GRAFCET porte de garage → SCL

Énoncé. Une impulsion télécommande (`bp_tc`) ouvre la porte ; en haut, temporisation de 30 s puis fermeture automatique ; pendant la fermeture, la cellule `obstacle` fait rouvrir ; fins de course `fdc_haut` , `fdc_bas` .

Corrigé

GRAFCET (à dessiner d'abord — c'est lui qui est noté) :



Implémentation SCL (FB, avec un `TON` nommé `tempo` en Static) :

```

CASE #etape OF
  0: // FERMÉE
    #monter := FALSE; #descendre := FALSE;
    IF #front_tc THEN #etape := 10; END_IF;

  10: // OUVERTURE
    #monter := TRUE; #descendre := FALSE;
    IF #fdc_haut THEN #etape := 20; END_IF;

  20: // OUVERTE (tempo de fermeture auto)
    #monter := FALSE; #descendre := FALSE;
    #tempo(IN := TRUE, PT := T#30s);
    IF #tempo.Q OR #front_tc THEN
      #tempo(IN := FALSE);           // réarmer la tempo en la quittant
      #etape := 30;
    END_IF;

  30: // FERMETURE
    #descendre := TRUE; #monter := FALSE;
    IF #obstacle OR #front_tc THEN
      #etape := 10;                 // priorité : on ROUVRE
    ELSIF #fdc_bas THEN
      #etape := 0;
    END_IF;
END_CASE;

// hors CASE : sécurité absolue, quel que soit l'état
IF #monter AND #descendre THEN    // ne doit JAMAIS arriver
  #monter := FALSE; #descendre := FALSE; #etape := 0;
END_IF;

```

Points de correction : une étape = un cas, les actions écrites dans **chaque** étape (pas d'action fantôme qui « reste collée ») ; l'ordre des conditions en étape 30 — l'obstacle est testé **avant** `fdc_bas` (priorité à la sécurité) ; la tempo réarmée en quittant l'étape 20 ; le verrouillage final monter/descendre.

Exercice 4 — Écran HMI

Énoncé. Écran avec bouton MARCHE, voyant moteur, champ consigne vitesse, alarme « défaut thermique ».

Corrigé (démarche attendue, à réaliser dans WinCC)

1. **Variables HMI** liées aux variables API : `cmd_marche` (Bool), `etat_moteur` (Bool), `consigne_vitesse` (Int, limites 0..1500), `defaut_thermique` (Bool).
2. **Bouton MARCHE** : événements « Appuyer » → mise à 1 de `cmd_marche`, « Relâcher » → mise à 0 (bouton à impulsion). **Piège de conception** : ne jamais faire un bouton HMI qui « reste à 1 » — si la liaison tombe, l'ordre reste bloqué. La logique de maintien appartient à l'automate (auto-maintien de l'exercice 1), pas à l'écran.

3. **Voyant** : un cercle dont l'apparence est animée par `etat_moteur` — et ce doit être le **retour réel** (contact du contacteur), pas la recopie de la commande : un voyant qui affiche « ce qu'on a demandé » au lieu de « ce qui se passe » masque les pannes.
 4. **Champ d'E/S** consigne : format numérique, **limites min/max déclarées** (l'automate revalide de son côté — défense en profondeur).
 5. **Alarme TOR** sur `defaut_thermique` : classe « Errors », texte explicite (« Défaut thermique moteur M1 — vérifier relais F2 »), acquittement requis ; fenêtre des alarmes sur l'écran.
 6. Test : simulateur HMI + PLCSIM ensemble, scénario complet (marche, changement de consigne, déclenchement du défaut, acquit).
-

Auto-évaluation avant le TP 3

Sans notes : justifier NF pour un AU ; dessiner l'auto-maintien ; expliquer front vs niveau dans un FB appelé chaque cycle ; différence FC/FB/DB ; pourquoi la logique de sécurité vit dans l'automate et jamais dans l'HMI.

 Passe au **TP 3 — Station de remplissage**.

FORMATION EMBARQUÉE

TD 08 Schneider

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 08 — Schneider : énoncés et corrigés détaillés

Cible : Machine Expert Basic (M221 simulé) pour les exercices 1 et 3, Control Expert (ou CODESYS) pour le ST et le SFC. Les concepts du TD 07 sont supposés acquis — on ne re-explique pas l'auto-maintien.

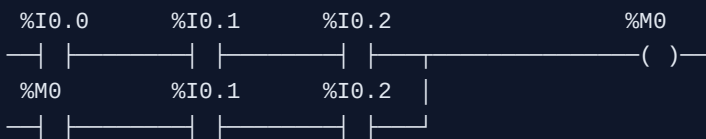
Exercice 1 — Démarrage étoile/triangle

Énoncé. Moteur en étoile/triangle : au départ, contacteur ligne **KM1** + contacteur étoile **KM2** pendant 5 s, puis passage en triangle **KM3**. Verrouillage KM2/KM3 impératif. Boutons **bp_marche** (%I0.0), **bp_arret** (%I0.1, NF câblé), défaut thermique **f_therm** (%I0.2, NF câblé).

Corrigé (LADDER M221)

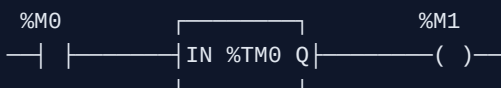
Objets : **%M0** = marche demandée ; **%TM0** : type TON, pré-réglage 5 s (**%TM0.P := 50** avec base 100 ms) ; **%M1** = tempo écoulée.

Réseau 1 — Auto-maintien de la demande :



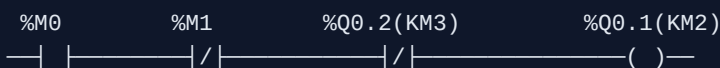
(arrêt et thermique en NF câblé : contact fermé = tout va bien.)

Réseau 2 — Temporisation : **%M0** lance **%TM0** (TON, 5 s) ; sortie du bloc → **%M1**.

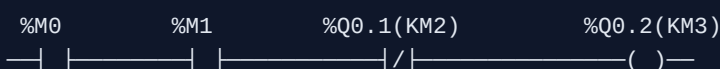


Réseau 3 — Contacteur ligne : **%Q0.0 (KM1) := %M0**.

Réseau 4 — Étoile (KM2) : marche ET tempo non écoulée ET triangle retombé :



Réseau 5 — Triangle (KM3) : marche ET tempo écoulée ET étoile retombée :



Points de correction : 1. Verrouillage croisé KM2/KM3 dans le programme ET, en vrai, aussi par contacts auxiliaires câblés : étoile + triangle fermés ensemble = court-circuit franc. Le logiciel ne suffit jamais seul pour ce risque. 2. Le passage étoile → triangle transite par un état où les deux sont ouverts

(chaque bobine attend la retombée de l'autre) — c'est voulu. 3. Arrêt et thermique coupent tout **via la demande** %M0 : une seule chaîne d'arrêt, pas trois copies.

Exercice 2 — Bloc **Alternance_Pompes** en ST

Énoncé. Deux pompes ; à chaque demande, démarrer la moins utilisée (compteurs d'heures) ; si la pompe choisie est en défaut, basculer sur l'autre.

Corrigé (DFB Control Expert / FB CODESYS)

Interface : Inputs **demande** , **default_p1** , **default_p2** (Bool) ; Outputs **cmd_p1** , **cmd_p2** , **alarme_aucune_pompe** (Bool) ; Statics **heures_p1** , **heures_p2** (DINT, en secondes), **demande_prec** (Bool), **choix_p1** (Bool), **tick** (instance de TON).

```
(* 1. Choix au FRONT MONTANT de la demande : on ne rebascule pas
   en cours de fonctionnement au fil des heures qui s'accumulent *)
IF demande AND NOT demande_prec THEN
    choix_p1 := (heures_p1 <= heures_p2);
END_IF;
demande_prec := demande;

(* 2. Report automatique sur défaut de la pompe choisie *)
IF demande THEN
    IF choix_p1 AND default_p1 AND NOT default_p2 THEN
        choix_p1 := FALSE;
    ELSIF NOT choix_p1 AND default_p2 AND NOT default_p1 THEN
        choix_p1 := TRUE;
    END_IF;
END_IF;

(* 3. Commandes — une pompe en défaut n'est JAMAIS commandée *)
cmd_p1 := demande AND choix_p1 AND NOT default_p1;
cmd_p2 := demande AND NOT choix_p1 AND NOT default_p2;
alarme_aucune_pompe := demande AND default_p1 AND default_p2;

(* 4. Comptage d'usure : +1 s par seconde de marche, via un TON auto-relancé *)
tick(IN := NOT tick.Q, PT := t#1s);
IF tick.Q THEN
    IF cmd_p1 THEN heures_p1 := heures_p1 + 1; END_IF;
    IF cmd_p2 THEN heures_p2 := heures_p2 + 1; END_IF;
END_IF;
```

Points de correction : le choix figé au front (sinon les pompes « papillonnent » quand les compteurs se croisent) ; le défaut qui inhibe la commande même si la logique de choix se trompait ; l'alarme « aucune pompe disponible » ; les compteurs en Static (rémanents à déclarer si l'usure doit survivre à une coupure — le préciser vaut des points).

Exercice 3 — Table d'échange Modbus + lecture Python

Énoncé. Documenter une table de 10 mots (états, mesures, commandes, mot de vie) et la lire/écrire depuis un PC.

Corrigé

La table (le livrable principal — c'est un document contractuel) :

Mot	Adresse Modbus	Sens	Contenu	Codage
%MW100	100	API → PC	Mot d'état : b0=auto, b1=marche, b2=défaut, b3=niveau haut	bits
%MW101	101	API → PC	Niveau cuve	0..1000 = 0..100,0 % (×10)
%MW102	102	API → PC	Débit pompe	L/min ×10
%MW103	103	API → PC	Code défaut	0=aucun, 1=thermique, 2=niveau...
%MW104	104	API → PC	Mot de vie : compteur +1/s	0..65535, boucle
%MW105	105	PC → API	Mot de commande : b0=marche, b1=acquit défauts	bits
%MW106	106	PC → API	Consigne niveau	% ×10, borné 0..1000
%MW107-109	107-109	—	Réserve (prévoir l'extension)	0

Conventions à énoncer : valeurs analogiques en **entiers ×10** (pas de flottants — un Real occuperait 2 mots avec un ordre dépendant de l'équipement) ; zones API → PC et PC → API **disjointes** (jamais les deux côtés écrivains du même mot) ; le **mot de vie** permet au PC de détecter un automate figé même quand la liaison TCP reste « ouverte ».

Côté PC (pymodbus ≥ 3.x) :

```

from pymodbus.client import ModbusTcpClient
import time

client = ModbusTcpClient("192.168.1.50", port=502)
client.connect()

vie_prec = None
for _ in range(5):
    rr = client.read_holding_registers(address=100, count=5)
    if rr.isError():
        print("erreur Modbus :", rr)
        break
    etat, niveau, debit, default, vie = rr.registers
    print(f"marche={bool(etat & 0x02)} default={bool(etat & 0x04)} "
          f"niveau={niveau/10:.1f}% vie={vie}")
    if vie_prec is not None and vie == vie_prec:
        print("ALERTE : mot de vie figé, automate en panne ?")
    vie_prec = vie
    time.sleep(1)

client.write_register(address=106, value=750) # consigne 75,0 %
client.write_register(address=105, value=0x0001) # bit marche
client.close()

```

Points de correction : la table documentée avec codages et bornes (sans elle, l'exercice vaut zéro : Modbus ne transporte que des mots — le sens est dans le document) ; la surveillance du mot de vie ; le test `isError()` .

Exercice 4 — Cycle de lavage en SFC (Control Expert)

Énoncé. Prélavage 2 min → lavage 5 min → rinçage ×2 → essorage 3 min ; pause/reprise ; abandon avec vidange sécurisée.

Corrigé (structure du SFC + mécanismes clés)



Mécanismes demandés :

- **Compteur de rinçages** : incrémenté à l'entrée de l'étape 4 (action au franchissement, qualificatif P1 chez Schneider), la divergence en OU après vidange reboucle si `nb_rincages < 2`.
- **Pause/reprise** : ne PAS ajouter des transitions « pause » partout — on fige les actions : chaque sortie est conditionnée par `NOT pause` (les actions de niveau `N` passent par une logique commune), et les temporisations utilisent des TON dont l'entrée `IN` est `etape_active AND NOT pause` : la tempo se **suspend** avec la pause.
- **Abandon** : une **séquence dédiée** (étapes 90-91 : arrêt chauffage et moteur → vidange jusqu'à `niveau_bas` → retour à 0), atteinte depuis n'importe où. En SFC Control Expert, l'abandon se traite proprement par un forçage de chaîne ou une transition source depuis une étape englobante ; l'implémenter par « IF abandon THEN etape := 90 » hors du CASE est aussi accepté en version ST.
- **Sécurité transversale hors SFC** : porte ouverte ⇒ moteur et chauffage coupés **immédiatement**, quelle que soit l'étape — cette ligne vit en logique câblée/LADDER en aval des sorties, pas dans la séquence. On ne confie jamais une coupure de sécurité à une séquence qui peut être bloquée dans une étape.

Points de correction : abandon = séquence (on ne « téléporte » pas à Init avec la cuve pleine d'eau chaude) ; tempos suspendues en pause ; sécurité porte hors séquence ; compteur remis à zéro à l'étape 0.

Auto-évaluation avant le TP 4

Sans notes : pourquoi le verrouillage étoile/triangle doit exister aussi en câblé ; les 4 zones de données Modbus et leurs codes fonction ; à quoi sert un mot de vie ; pourquoi une pause ne s'implémente pas en dupliquant des transitions ; où vit une coupure de sécurité (et où elle ne vit pas).

 Passe au [TP 4 — Gestion de cuve](#).

FORMATION EMBARQUÉE

TD 10 Stm32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 10 — STM32 : énoncés et corrigés détaillés

Cible : Nucleo-F411RE (adapter les broches sinon). Les corrigés montrent le code utilisateur à placer entre les balises `USER CODE` d'un projet CubeMX ; la configuration CubeMX est décrite à chaque fois.

Exercice 1 — Chenillard cadencé par TIM3, vitesse par ADC

Énoncé. 4 LED avancent au rythme d'une interruption TIM3 ; le potentiomètre (ADC) règle la vitesse. Zéro `HAL_Delay` dans la boucle.

Corrigé

Config CubeMX : PA5-PA8 en GPIO_Output ; PA0 en ADC1_IN0 (résolution 12 bits, mode continu OFF) ; TIM3 : horloge interne, PSC = 9999, ARR = 999 (à 100 MHz APB1 timer clock → $100 \text{ MHz} / 10000 / 1000 = 10 \text{ Hz}$ de base), interruption « update » activée dans NVIC.

```

/* USER CODE BEGIN PV */
static const uint16_t LED_PINS[4] = {GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7, GPIO_PIN_8};
static volatile uint8_t index_led = 0;      // écrit en ISR, lu partout
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
// ISR de débordement TIM3 : avancer le chenillard. COURTE : 3 écritures GPIO.
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3) {
        HAL_GPIO_WritePin(GPIOA, LED_PINS[index_led], GPIO_PIN_RESET);
        index_led = (index_led + 1u) & 0x03u;    // modulo 4 par masque
        HAL_GPIO_WritePin(GPIOA, LED_PINS[index_led], GPIO_PIN_SET);
    }
}

static uint16_t lire_pot(void)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    uint16_t v = (uint16_t)HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return v;                                // 0..4095
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim3);              // démarrer le timer + IRQ
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    // La vitesse se règle en modifiant ARR : période = (PSC+1)(ARR+1)/f_tim.
    // pot 0..4095 → ARR 199..4095+199 → ~2 Hz à ~25 Hz d'avancement.
    uint16_t pot = lire_pot();
    __HAL_TIM_SET_AUTORELOAD(&htim3, 199u + pot);
    HAL_Delay(50);    // toléré ICI : ne cadence rien, il espace les lectures ADC
}
/* USER CODE END WHILE */

```

Points de correction : le rythme vient du **matériel** (TIM3), pas d'une attente logicielle — coupe le debugger en pause : le chenillard continue ; **volatile** sur **index_led** ; modification d' **ARR** à chaud (le registre « auto-reload » est fait pour ça).

Exercice 2 — Console UART : réception IT + ring buffer + commandes

Énoncé. Réception par interruption dans un tampon circulaire ; commandes **pwm <0-100>** et **status** , avec écho.

Corrigé (extraits essentiels — réutilise le ring buffer du TD 01)

Config CubeMX : USART2 115200 8N1, interruption activée ; TIM2 CH1 (PA5) en PWM, PSC/ARR pour 1 kHz avec ARR = 999 (rapport en ‰ $\approx \% \times 10$).


```

/* USER CODE BEGIN PV */
static RingBuffer rb_rx;           // le ring buffer du TD 01, tel quel
static uint8_t octet_rx;          // tampon d'1 octet pour la HAL
static char ligne[32];
static uint8_t ligne_pos = 0;
static uint8_t pwm_pct = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
// ISR : UN octet reçu → on le pousse dans le ring buffer et on réarme.
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
        (void)rb_put(&rb_rx, octet_rx);           // plein ? octet perdu, tant pis
        HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // RÉARMER : sinon plus rien
    }
}

static void traiter_ligne(const char *l)
{
    int v;
    if (sscanf(l, "pwm %d", &v) == 1 && v >= 0 && v <= 100) {
        pwm_pct = (uint8_t)v;
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, v * 10u); // ARR=999 → 10%
        printf("OK pwm=%d%\r\n", v);
    } else if (strcmp(l, "status") == 0) {
        printf("pwm=%u%% uptime=%lums\r\n", pwm_pct, (unsigned long)HAL_GetTick());
    } else if (l[0] != '\0') {
        printf("ERR commande inconnue\r\n");
    }
}

// Redirection de printf vers l'UART (newlib de ST)
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 10);
    return ch;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
rb_init(&rb_rx);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // amorcer la réception
printf("console prete\r\n");
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    uint8_t c;
    while (rb_get(&rb_rx, &c)) { // vider ce qui est arrivé
        HAL_UART_Transmit(&huart2, &c, 1, 10); // écho
        if (c == '\r' || c == '\n') {
            ligne[ligne_pos] = '\0';

```

```

        traiter_ligne(ligne);
        ligne_pos = 0;
    } else if (ligne_pos < sizeof ligne - 1) {    // TOUJOURS border
        ligne[ligne_pos++] = (char)c;
    }
}
// ...la boucle reste disponible pour le reste de l'application...
}
/* USER CODE END WHILE */

```

Points de correction : le réarmement dans le callback (l'oubli = « je ne reçois que le premier caractère », bug archi-classique) ; **l'ISR ne parse pas** — elle stocke, la boucle parse (règle « ISR courte ») ; la ligne bornée ; la validation `0..100` avant d'appliquer.

Exercice 3 — Mesure de fréquence par Input Capture

Énoncé. Mesurer la fréquence d'un signal carré (reboucler le PWM de l'exercice 2 sur l'entrée !), affichage UART.

Corrigé

Config CubeMX : TIM4 CH1 (PB6) en « Input Capture direct mode », front montant, PSC = 99 (→ compteur à 1 MHz : 1 tick = 1 µs), ARR = 65535, interruption capture activée. Relier PA5 (PWM) à PB6 par un fil.

```

/* USER CODE BEGIN PV */
static volatile uint32_t periode_us = 0;
static volatile uint8_t mesure_prete = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    static uint16_t capture_prec = 0;
    if (htim->Instance == TIM4 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
        uint16_t capture = (uint16_t)HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
        // Soustraction sur uint16_t : correcte MÊME si le compteur a débordé
        // entre les deux fronts (arithmétique modulo 65536 – cf. millis() !)
        periode_us = (uint16_t)(capture - capture_prec);
        capture_prec = capture;
        mesure_prete = 1;
    }
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_1);
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    if (mesure_prete) {
        mesure_prete = 0;
        uint32_t p = periode_us;           // copie locale
        if (p > 0)
            printf("f = %lu Hz\r\n", (unsigned long)(1000000UL / p));
    }
    HAL_Delay(200);
}
/* USER CODE END WHILE */

```

Points de correction : le prescaler choisi pour un tick « rond » (1 μ s : le calcul de fréquence devient trivial et la plage mesurable — période max 65,535 ms $\rightarrow$ ~15 Hz mini — doit être **énoncée**) ; la soustraction non signée qui absorbe le débordement ; la mesure faite par le **matériel** au front près (aucun jitter logiciel, contrairement à un comptage dans une ISR GPIO).

Exercice 4 — Provoquer et analyser un HardFault

Énoncé. Déréférence un pointeur invalide, observe, explique.

Corrigé

```

volatile uint32_t *p = (uint32_t *)0xDEADBEEF; // adresse non mappée
*p = 42; // 💥 BusFault  $\rightarrow$  HardFault

```

Ce qu'on observe : le débogueur s'arrête dans `HardFault_Handler()` (boucle infinie générée par CubeMX). Le **Fault Analyzer** de CubeIDE (fenêtre *Fault Analyzer* en session de debug) décode les registres système :

- `CFSR` : le bit **PRECISERR** (BusFault précis) est levé ;
- `BFAR` contient **0xDEADBEEF** — l'adresse fautive exacte ;
- la pile empilée par le Cortex-M donne le **PC fautif** → double-clic ramène à la ligne `*p = 42;` .

Explication attendue : à l'exception, le Cortex-M empile automatiquement R0-R3, R12, LR, PC, xPSR ; les registres de faute (CFSR/HFSR/BFAR/MMFAR) disent *quoi* et *où*. C'est toute la différence avec l'AVR, qui redémarre sans laisser d'indice. En production, on écrit un handler qui sauvegarde ces registres (en RAM non initialisée ou en flash) puis redémarre — le crash devient diagnosticable après coup.

Variantes à essayer : lecture à une adresse non alignée castée en `uint32_t*` (selon la config, UsageFault UNALIGNED), appel d'un pointeur de fonction nul, débordement de pile (récursion) — chacun signe différemment dans le CFSR.

Auto-évaluation avant le TP 5

Sans notes : calculer PSC/ARR pour une fréquence donnée ; expliquer le réarmement de `HAL_UART_Receive_IT` ; pourquoi l'Input Capture bat toute mesure logicielle ; citer 3 registres de diagnostic d'un HardFault et leur rôle.

➡ Passe au **TP 5 — Station météo STM32 + FreeRTOS**.